SE3040 – Application Frameworks

BSc (Hons) in Information Technology Specialized in Software Engineering

Year 03 Semester 01

Deployment Report

Project Title: EcoLife – Sustainable Living Platform

SE Group Number – SE-62

	Registration Number	Student Name	Student E-mail Address	Contact Number
1.	IT23668690	EDIRISINGHE D V M	it23668690@my.sliit.lk	0752903525
2.	IT23587038	PERERA J.A.A.H.D	it23587038@my.sliit.lk	0717466262
3.	IT23733862	RATHNASEKARA R K A S T	it23733862@my.sliit.lk	0711396048
4.	IT23591134	MUTHUNILMI T.G.K	it23591134@my.sliit.lk	0761620836

Table of Contents

1. Introduction 3
2. Deployment Overview 4
3. Backend Deployment 5
3.1.Platform: Render 5
3.2.Setup Steps: 5
3.3.Result: 5
4. Frontend Deployment 6
4.1.Platform: Vercel 6
4.2.Setup Steps: 6
4.3.Result: 6
5. Environment Variables 7
5.1.Backend: 7
5.2.Frontend: 8
6. Live URLs 8
7. Deployment Evidence 9
7.1.Screenshot of backend deployment (Render dashboard) 9
7.2.Screenshot of frontend deployment (Vercel dashboard) 12
7.3.Screenshot of live frontend application 14
7.4.Screenshot of API response from deployed backend 14
8. Conclusion 15

1. Introduction

EcoLife is a full-stack web application developed to support sustainable living and promote environmentally responsible behavior in alignment with the United Nations Sustainable Development Goal 12: Responsible Consumption and Production.

The system provides an integrated platform that allows users to monitor and reduce their environmental impact through multiple features such as Waste Tracking, Energy Monitoring, Marketplace for reusable items, and Recycling Center management. In addition, the Waste Tracker module incorporates AI-based image analysis to identify waste types from uploaded images, enhancing user convenience and promoting accurate waste categorization.

The application is built using the MERN stack, with React (Vite) for the frontend, Node.js and Express.js for the backend, and MongoDB for data storage. The system also integrates third-party services for advanced functionalities such as image processing, CO_2 impact calculation, AI based Energy saving tips suggestion and location-based services.

EcoLife follows a modular architecture with role-based access control, allowing both administrators and regular users to interact with the system securely. Users can track their daily waste generation, monitor energy usage, participate in a reuse marketplace, and locate nearby recycling centers, while administrators manage system data and overall sustainability metrics.

This document presents the deployment process of the EcoLife system, including the platforms used, configuration steps, environment variables, and live system access details. The deployment ensures that the application is accessible in a real-world environment, supporting scalability, reliability, and continuous usage beyond local development.

2. Deployment Overview

The EcoLife application was deployed using a cloud-based architecture to ensure accessibility, scalability, and reliable performance in a real-world environment. The system follows a full-stack deployment model where the frontend, backend, and database are hosted on separate cloud platforms.

- **Frontend:** Deployed using Vercel, enabling fast and optimized delivery of the React (Vite) application with global content distribution.
- **Backend:** Deployed using Render as a web service to host the Node.js and Express.js RESTful API, ensuring secure and continuous server-side operations.
- **Database:** MongoDB Atlas was used as the cloud-hosted database to provide scalable and reliable data storage with secure access management.

This deployment architecture allows the EcoLife system to operate independently of a local environment, making it accessible from any location via the internet. It also supports real-time interaction between the frontend and backend, ensuring seamless data processing, secure API communication, and consistent system performance under user load.

3. Backend Deployment

3.1.Platform: Render

3.2.Setup Steps:

1. The project was pushed to a GitHub repository.
2. A new Web Service was created on the Render platform.
3. As root path select backend folder from GitHub repository and select on commit as auto deploy option.
4. The following configurations were applied:
    - **Build Command**: npm install
    - **Start Command:** node index.js
5. Required environment variables such as the MongoDB connection string (MONGO_URI) and JWT secret (JWT_SECRET) were securely configured in the platform dashboard.
6. The service was deployed successfully, and a public URL was generated for accessing the backend API.

3.3.Result:

- The backend API is running successfully in the production environment
- All REST API endpoints are accessible via the deployed URL
- Authentication, authorization, and CRUD operations function correctly
- The server remains stable and handles requests reliably

4. Frontend Deployment

4.1.Platform: Vercel

4.2.Setup Steps:

1. The project was pushed to a GitHub repository.
2. A new project was created on the Vercel platform.
3. As root path select frontend folder from GitHub repository add vercel bot to GitHub to track latest changes.
4. The following configurations were applied:
    - **Framework: Vite**
    - **Build Command: npm run build**
    - **Output Directory: dist**
5. The frontend application was successfully built and deployed, generating a public URL for access.

4.3.Result:

- The frontend application loads successfully in the production environment
- All UI components render correctly and are fully responsive
- Navigation and routing function properly without errors
- API communication with the deployed backend works correctly
- The application provides a smooth and consistent user experience

5. Environment Variables

The following environment variables were configured during deployment to ensure secure and proper functioning of the EcoLife application:

5.1.Backend:

- **MONGO_URI** – The MongoDB Atlas connection string used to establish a secure database connection between your Render backend and the cloud database.
- **JWT_SECRET** – A secret cryptographic key used for signing and validating JSON Web Tokens (JWT) to ensure secure user authentication and authorization.
- **JWT_EXPIRE** – Defines the duration for which a user's login session remains valid (e.g., 7d for seven days) before they are required to log in again.
- **PORT** – Specifies the logical port on which the Express server will listen for incoming traffic (defaults to 10000 on Render and 5000 locally).
- **NODE_ENV** – Indicates the current environment of the application (development or production), which affects error logging, performance optimizations, and security settings.
- **FIREBASE_PROJECT_ID** – The unique identifier for your Firebase project, required to initialize the Firebase Admin SDK for backend tasks.
- **FIREBASE_PRIVATE_KEY** – The secret private key from your Firebase service account used to authenticate your server's requests to Firebase services.
- **FIREBASE_CLIENT_EMAIL** – The service account email address associated with your Firebase project, used for identity verification during SDK initialization.
- **FIREBASE_STORAGE_BUCKET** – The URL of your Firebase Cloud Storage bucket, used for storing and retrieving user-uploaded images (like waste photos or marketplace items).
- **GEMINI_API_KEY** – The API key for Google's Gemini AI, used to power features like the smart energy-saving tips and AI-driven suggestions in the app.
- **GOOGLE_CREDS** – The complete Google Cloud Service Account credentials stored as a JSON string, enabling the Google Vision API to classify waste images for the recycling tracker.
- **CLIENT_URL** – The URL(s) of the frontend application (e.g. - Vercel deployment) that are authorized by the backend's CORS (Cross-Origin Resource Sharing) policy to communicate with the API and access protected resources.

5.2.Frontend:

- **VITE_GOOGLE_MAPS_API_KEY** – A unique API key obtained from the Google Cloud Console used to authenticate and load Google Maps services, enabling the "Recycle Centers" map and location-based search functionality.
- **VITE_API_URL** – The primary entry-point URL for the backend server (e.g., Render deployment) that the frontend application uses to communicate with the API for authentication, data retrieval, and AI processing.

All sensitive values were securely stored within the respective deployment platform dashboards (Render and Vercel) and were not exposed in the source code or version control system. This approach ensures data security and prevents unauthorized access to critical configuration details.

6. Live URLs

- **Frontend Application:** https://eco-life-pi.vercel.app/
- **Backend API:** https://ecolife-server.onrender.com

7. Deployment Evidence

The following screenshots will be included as evidence of successful deployment:

7.1.Screenshot of backend deployment (Render dashboard)

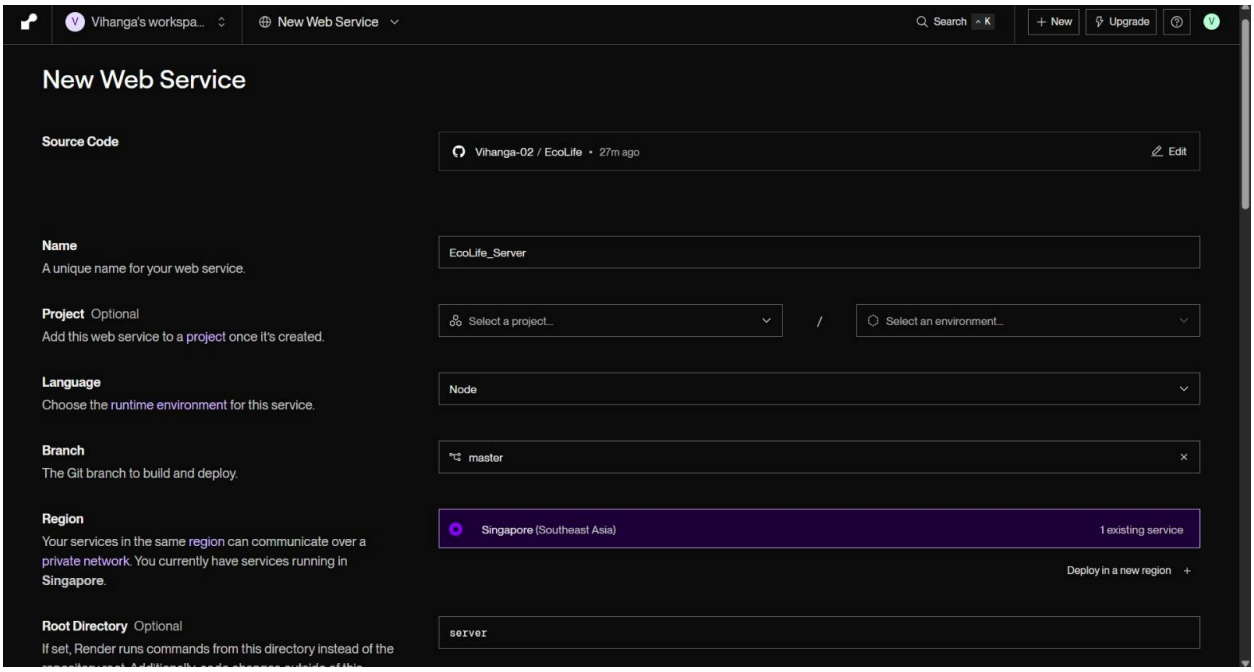

Figure 1: Creating a New Web Service on Render

This figure shows the configuration of the backend service on the Render platform, including repository selection, service name, branch, region, and root directory setup.

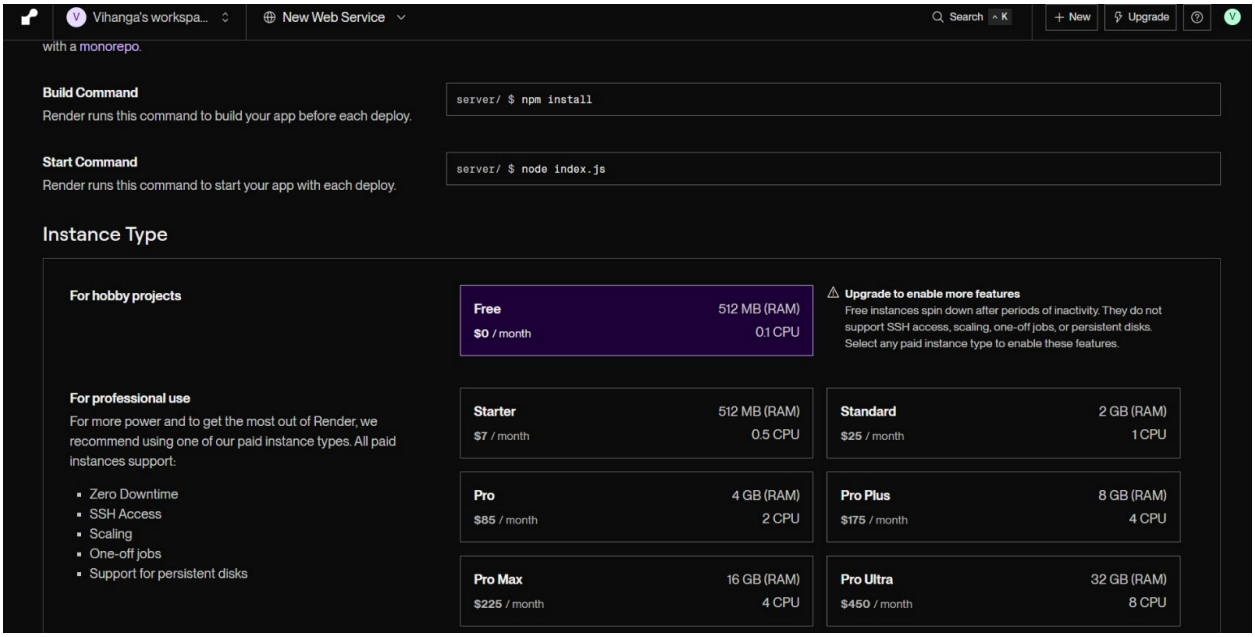

Figure 2:Build and Start Command Configuration

This figure illustrates the build and start commands used for deploying the backend service. The application is configured to install dependencies using npm install and start the server using node index.js.

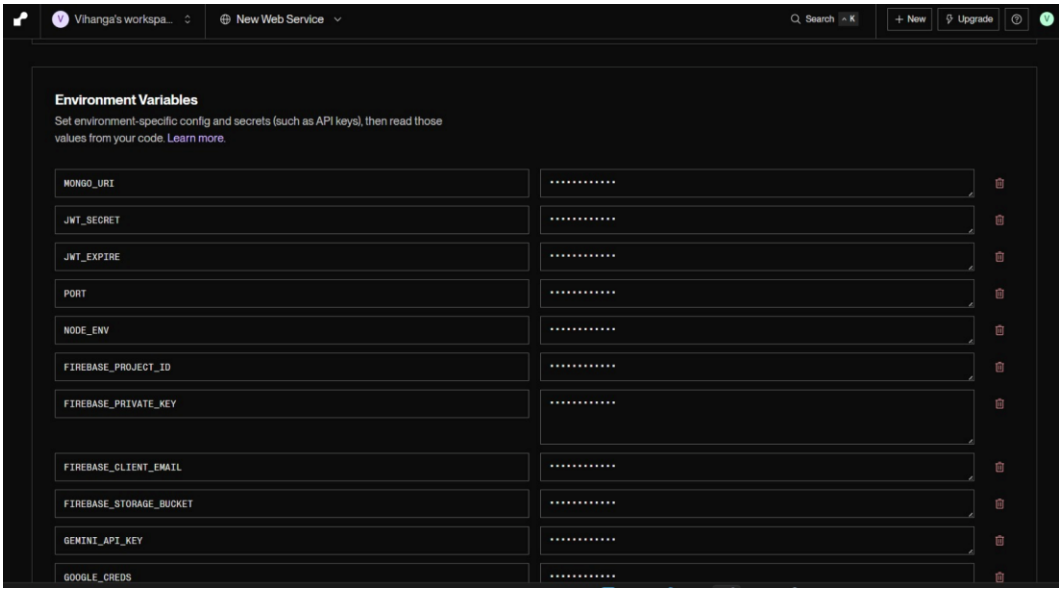

Figure 3:Environment Variable Configuration

This figure displays the environment variables configured in Render, including database connection (MONGO_URI), authentication secret (JWT_SECRET), Firebase configuration, and API keys. Sensitive values are securely stored and not exposed.

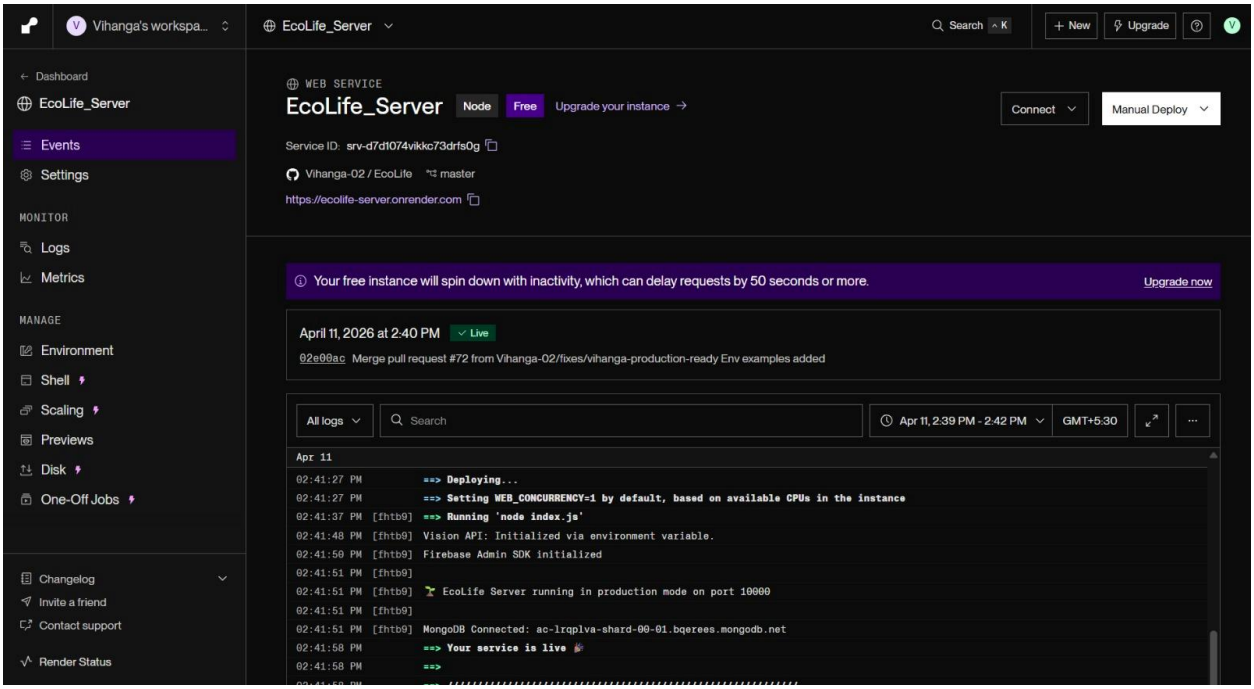

Figure 4:Backend Service Successfully Deployed

This figure shows the Render dashboard indicating that the EcoLife backend service is live and accessible via a public URL.

Figure 5: Update CLIENT_URL After deploying Frontend

Add Vercel deployment URL to Render backend service to create connection with Frontend and Backend

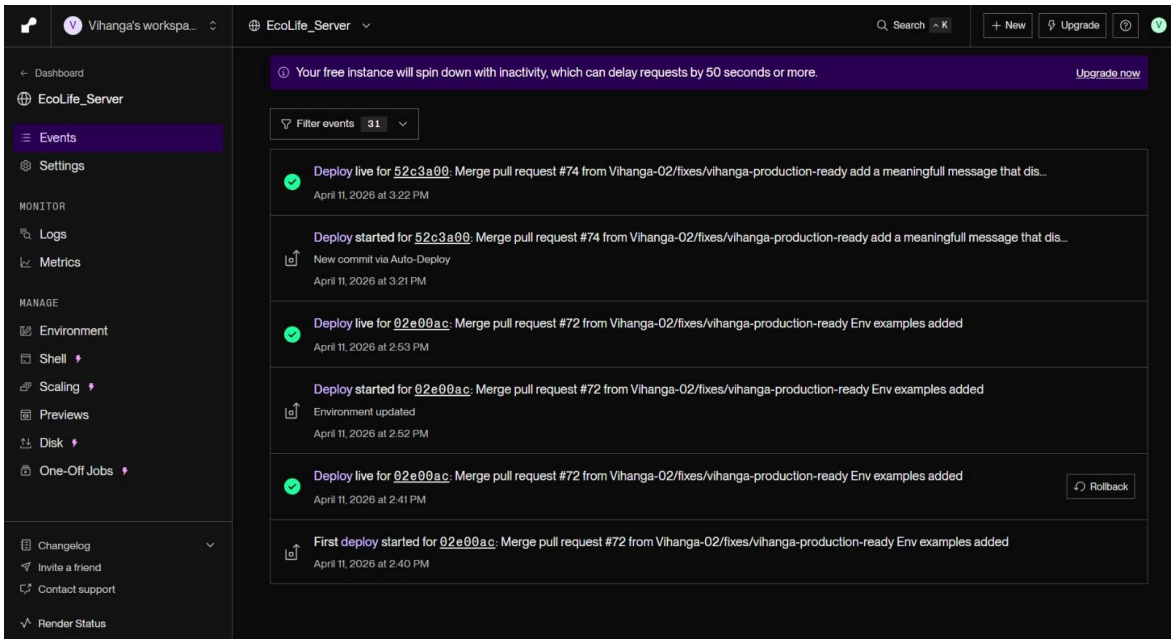

Figure 6:Deployment History and Status

This figure shows the deployment history of the backend service on Render. The deployments are automatically triggered through GitHub integration, and all builds were completed successfully, indicating a stable and continuously updated system.

7.2.Screenshot of frontend deployment (Vercel dashboard)

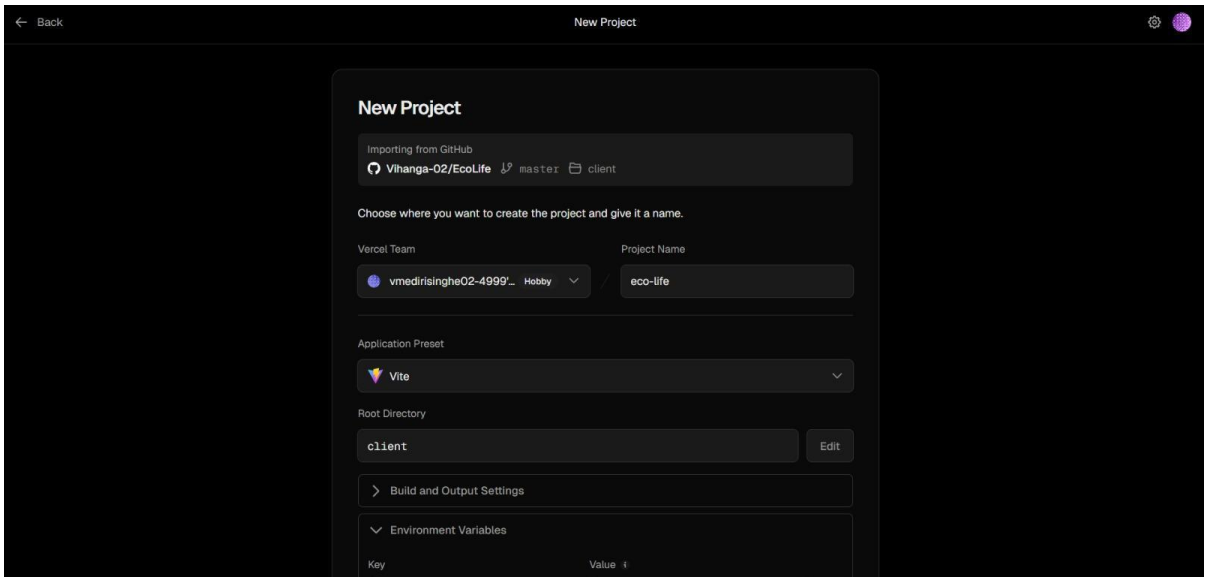

Figure 7:Frontend Deployment Setup on Vercel

This figure shows the initial configuration stage of deploying the EcoLife frontend application on Vercel. The project is imported from a GitHub repository, and key settings such as the project name, framework (Vite), and root directory are defined before initiating the deployment process

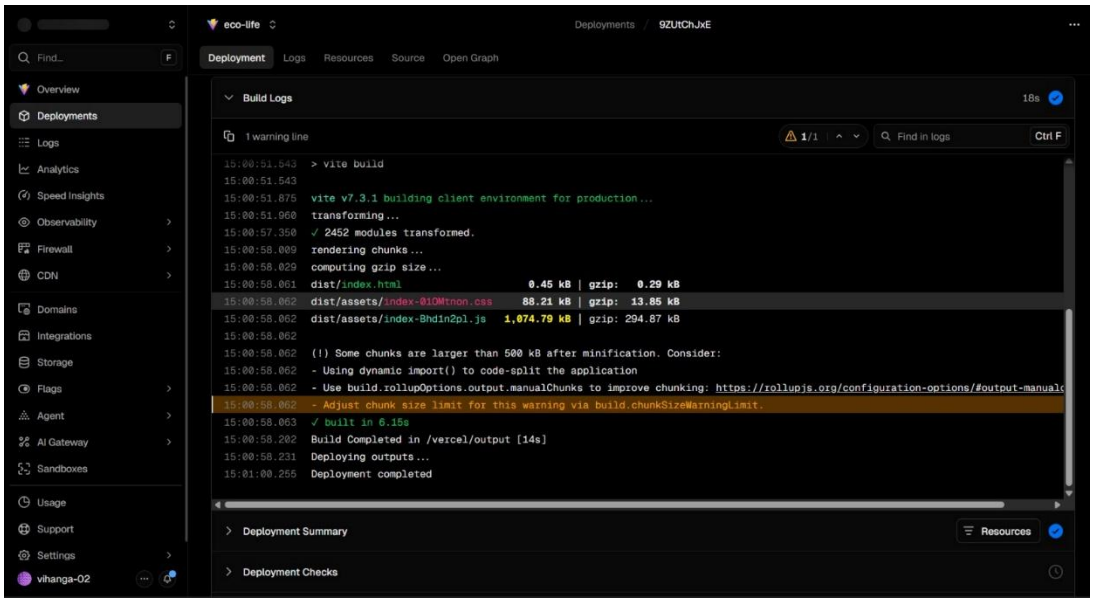

Figure 8: Frontend Build Logs on Vercel

This figure displays the build logs of the EcoLife frontend application during deployment on Vercel. The logs confirm that the project was successfully built using Vite and deployed without errors. A minor warning related to bundle size is observed; however, it does not affect the overall functionality of the application.

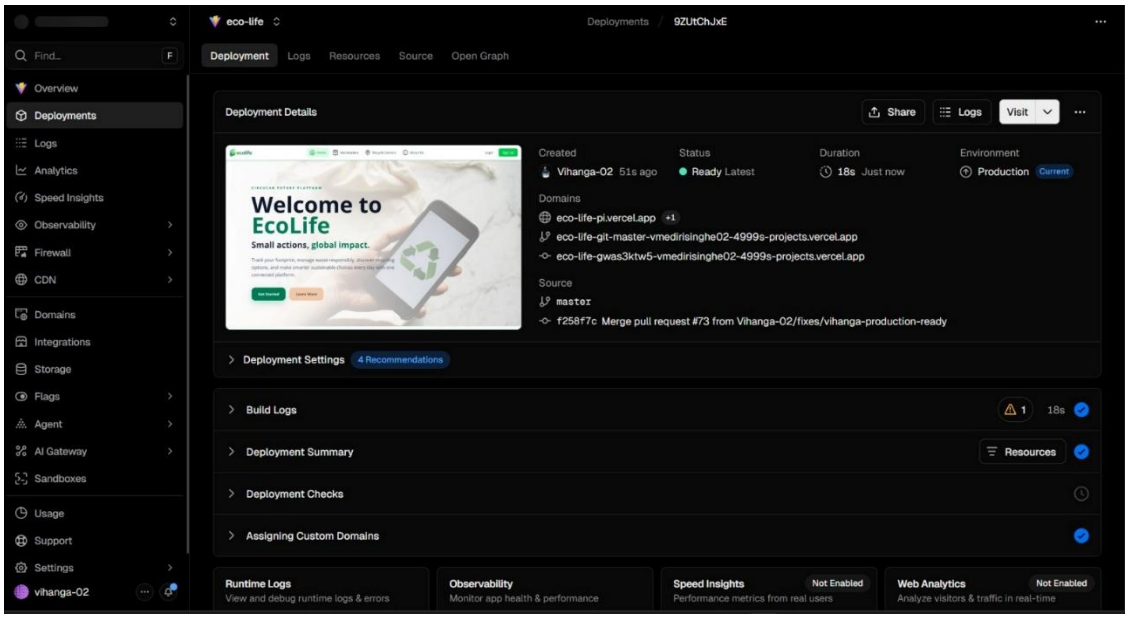

Figure 9: Frontend Deployment Status on Vercel

This figure shows the successful deployment of the EcoLife frontend application on Vercel. The deployment status is marked as “Ready,” indicating that the application is live and running in the production environment. The public domain URLs are displayed, confirming that the system is accessible to users.

7.3.Screenshot of live frontend application

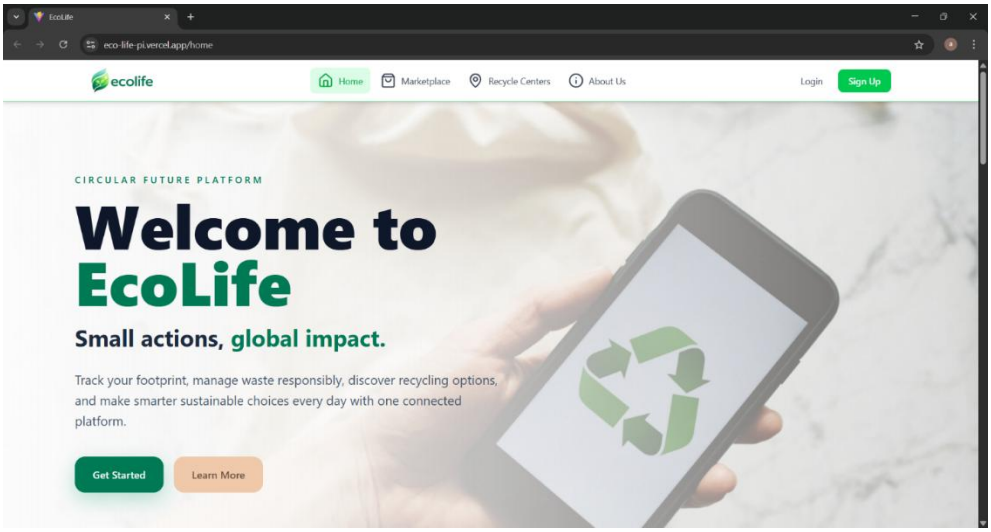

Figure 10: Live Frontend Application (EcoLife)

This figure shows the live EcoLife frontend application running in the production environment, accessed through the deployed Vercel URL. The user interface loads successfully, confirming that the frontend deployment is complete and functioning as expected

7.4.Screenshot of API response from deployed backend

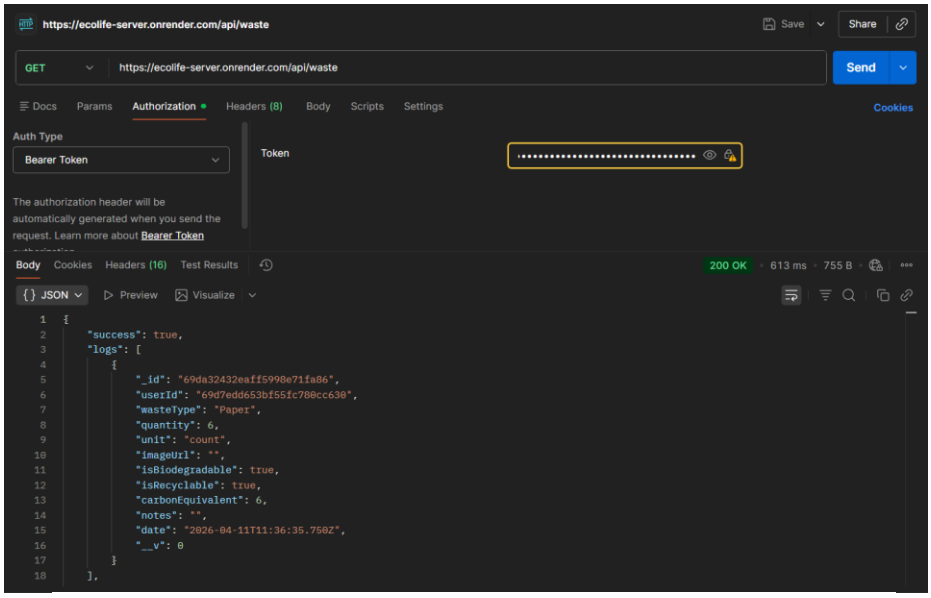

Figure 11: Successful API Response from Waste Endpoint

This figure shows a successful response from the /api/waste endpoint of the deployed EcoLife backend. The request was authenticated using a valid JWT token, and the system successfully retrieved waste log data from the database. This confirms that the backend API, database connectivity, and authentication mechanisms are functioning correctly in the production environment.

8. Conclusion

The deployment of the EcoLife application was successfully completed using modern cloud platforms, ensuring reliable and scalable system availability. The backend was deployed on Render, while the frontend was hosted on Vercel, with MongoDB Atlas used as the cloud database solution. This architecture enabled seamless communication between system components and provided global accessibility without the need for a local setup.

Throughout the deployment process, all critical configurations such as environment variables, authentication mechanisms, and database connections were properly implemented. The system was tested after deployment to verify functionality, including API responses, user authentication, and data retrieval operations.

The deployment evidence, including platform dashboards, build logs, live application interface, and API responses, confirms that the EcoLife system is fully operational in the production environment. The application demonstrated stability, correct functionality, and secure handling of user data.

In conclusion, the EcoLife system has been successfully deployed and is capable of supporting real-world usage, providing a reliable platform for promoting sustainable living practices through its integrated features such as Waste Tracking, Energy Monitoring, Marketplace, and Recycling Management.